

Modelagem, Implementação e Análise Comparativa de Estruturas em Árvore Especializadas

Kairo Henrique Ferreira Martins

Trabalho Prático Individual I — Algoritmos e Estruturas de Dados II

github.com/KairoHenrique/TP1-Estruturas-em-Arvores-Avancadas

Resumo

Este relatório estuda, implementa e compara cinco estruturas hierárquicas alternativas às árvores BST e AVL: Trie, Patricia (radix compacta), Splay, Treap e KD-Tree. Cada uma responde a um recorte distinto do problema de indexação — prefixos de strings, localidade de acesso, balanceamento probabilístico e geometria em $\mathbb{R}^k$. As implementações em Python compartilham API uniforme e instrumentação de comparações, rotações e tempo.

A avaliação experimental usa três universos de chaves ($n \in \{10^3, 5 \cdot 10^3, 10^4\}$): strings (Trie vs Patricia), chaves ordenáveis (Splay, Treap, BST e AVL) e pontos k -dimensionais (KD-Tree vs força bruta). A Patricia reduz nós em cerca de $3,7\times$ frente à Trie; a BST degenera na inserção ordenada; a Splay se beneficia de Zipf/localidade; a KD-Tree é cerca de $580\times$ mais rápida que força bruta em NN 2D. Nenhuma estrutura domina fora do seu domínio.

Palavras-chave: Trie, Patricia, Splay, Treap, KD-Tree, complexidade, avaliação experimental.

1 Introdução e fundamentação teórica

Estruturas hierárquicas organizam dados de modo que o custo de busca, inserção e remoção dependa da *forma* da árvore, e não apenas do volume de chaves. Árvores binárias de busca (BST) e árvores AVL resolvem o dicionário clássico sobre um universo totalmente ordenado: a BST oferece operações proporcionais à altura, linear no pior caso; a AVL impõe invariante de altura e garante $O(\log n)$ determinístico [Cormen et al., 2012]. Este trabalho parte desse referencial para estudar cinco estruturas *não convencionais*, cada uma otimizada para um recorte diferente do problema de indexação [Sedgewick and Wayne, 2011].

A motivação é prática. Conjuntos de strings compartilham prefixos (dicionários, roteamento, autocompletar); sequências de acesso exibem localidade temporal; aplicações geométricas consultam vizinhança em $\mathbb{R}^k$. Nesses cenários, BST e AVL são corretas, mas não exploram a estrutura da chave. As cinco árvores implementadas respondem a essas demandas com invariantes distintos.

1.1 Trie (árvore de prefixos)

Uma Trie representa um conjunto de strings ramificando um caractere por aresta. O caminho da raiz até um nó com marca de fim de palavra corresponde a uma chave. O invariante central é o **compartilhamento de prefixos**: duas palavras que coincidem nos primeiros ℓ símbolos compartilham os ℓ primeiros nós. No modelo de alfabeto constante, as operações dependem do comprimento L da chave, não de n . Anatomia do nó: mapa de filhos indexado pelo próximo símbolo e um bit de término. Problema-alvo: dicionários, autocompletar e busca por prefixo [Sedgewick and Wayne, 2011].

1.2 Árvore Patricia (radix tree compacta)

A Patricia comprime caminhos unários da Trie: cada aresta armazena uma *string*, não um único caractere [Morrison, 1968]. Inserir uma chave que diverge no meio de uma aresta **divide** o rótulo; remover pode **recompactar** um caminho unário. O invariante é o mesmo conjunto de chaves da Trie, com menos nós. O nó contém o rótulo da aresta incidente, o bit de término e o mapa de filhos pela primeira letra do rótulo. Problema-alvo: indexação compacta de strings e roteamento por prefixo, em que o espaço da Trie ingênua seria proibitivo.

1.3 Árvore Splay

Splay é uma BST autoajustável [Sleator and Tarjan, 1985]. Após cada acesso, o nó consultado sobe à raiz por rotações *zig*, *zig-zig* e *zig-zag*. Não há fator de balanceamento explícito: o invariante é o de BST, acrescido da heurística de que chaves frequentes permanecem rasas. A análise amortizada por potencial estabelece $O(\log n)$ por operação no caso médio amortizado; o pior caso pontual ainda pode ser linear. Problema-alvo: cargas com localidade temporal ou distribuição altamente desigual.

1.4 Árvore Treap

Treap combina BST nas chaves com heap nas **prioridades** aleatórias [Seidel and Aragon, 1996]. Cada nó (k, p) satisfaz: descendente esquerdo com chave $< k$, direito com chave $> k$, e filhos com prioridade $\leq p$ (heap máximo). Rotações restauram o heap após inserir ou remover. Com prioridades uniformes independentes, a árvore tem a mesma distribuição de uma BST aleatória: altura esperada $O(\log n)$. Problema-alvo: dicionário com balanceamento probabilístico e implementação mais simples que AVL.

1.5 KD-Tree

A KD-Tree particiona $\mathbb{R}^k$ alternando o eixo de corte [Bentley, 1975]. No nível d , o nó compara a coordenada $d \bmod k$. Filhos esquerdo e direito ocupam semiespaços. O invariante espacial permite **poda** em busca do vizinho mais próximo e em consultas por intervalo. Anatomia: ponto k -dimensional, eixo de corte, dois filhos. Problema-alvo: geometria computacional, GIS e reconhecimento de padrões — não substituição de BST para chaves escalares.

1.6 Diferenças conceituais em relação a BST e AVL

BST e AVL indexam um domínio ordenado unidimensional. Trie e Patricia indexam *sequências* e tornam a busca por prefixo nativa. Splay relaxa o balanceamento estático da AVL em troca de adaptação ao padrão de acesso. Treap recupera o $O(\log n)$ esperado sem manter alturas. KD-Tree abandona a ordem total: a “esquerda/direita” depende do eixo. Essas diferenças justificam os três grupos experimentais da Seção 5: as estruturas **não são intercambiáveis no mesmo universo de chaves**.

2 Projeto e implementação das estruturas

Todas as estruturas foram implementadas em Python, com API uniforme (`insert`, `search`, `delete`, `to_dot`) e um objeto `Metrics` que conta comparações, rotações e nós alocados. Baselines BST e AVL existem apenas para o grupo de chaves ordenáveis. O código completo permanece fora deste relatório [Martins, 2026].

2.1 Representação computacional

A Tabela 1 resume os campos de cada nó e o critério de organização da árvore.

Tabela 1: Anatomia dos nós e organização das cinco estruturas (e baselines).

Estrutura	Atributos do nó	Organização
Trie	children, is_end	um caractere por aresta
Patricia	edge, children, is_end	string compacta na aresta; <i>split</i> no mismatch
Splay / BST / AVL	key, left, right (+ height)	BST clássica
Treap	key, priority, left, right	BST + heap máximo
KD-Tree	point, axis, left, right	corte alternado em k dimensões

2.2 Operações e manutenção do invariante

Trie. A inserção percorre ou cria um filho por caractere e marca o terminal. A remoção desmarca o terminal e **poda** ramos sem filhos e sem fim de palavra. Operação específica: *starts_with* / *words_with_prefix*.

Patricia. A inserção calcula o prefixo comum entre a chave restante e o rótulo da aresta. Se o prefixo é parcial, cria-se um nó intermediário (Algoritmo 1). A remoção desmarca o terminal, elimina folhas inúteis e compacta nós unários que não são fim de palavra.

Splay. A rotina de splay é recursiva (*bottom-up*). A inserção splaya a chave (ou a última visitada) e recorta a árvore à esquerda/direita da nova raiz. A remoção splaya o alvo e junta a subárvore esquerda (splay do máximo) com a direita [Sleator and Tarjan, 1985].

Treap. Inserção BST seguida de rotações enquanto a prioridade do filho viola o heap. Remoção rotaciona o nó-alvo para baixo, sempre na direção do filho de maior prioridade, até que tenha zero ou um filho [Seidel and Aragon, 1996]. Prioridades são amostradas por gerador com semente apenas nos experimentos, para reprodutibilidade.

KD-Tree. A inserção desce pelo eixo $depth \bmod k$. O vizinho mais próximo explora primeiro o lado do ponto-consulta e só visita o outro lado se a hipersuperfície de corte intersecta a bola do melhor candidato. A *range query* é uma DFS com poda por intervalo. A remoção substitui o ponto pelo mínimo da dimensão de corte na subárvore adequada [Bentley, 1975].

Algorithm 1 Inserção na Patricia (radix compacta): divisão de aresta.

entrada: nó corrente u , sufixo restante s **saida:** árvore com s inserida, invariante de prefixos preservado

```
1: if  $s = \varepsilon$  then
2:   marque  $u$  como fim de palavra; return
3: end if
4:  $v \leftarrow$  filho de  $u$  cuja aresta começa com  $s[0]$ , ou nil
5: if  $v = \text{nil}$  then
6:   crie folha com rótulo  $s$  e marque fim de palavra
7: else
8:    $\ell \leftarrow$  comprimento do prefixo comum entre o rótulo de  $v$  e  $s$ 
9:   if  $\ell = |\text{rótulo}(v)|$  then
10:    insira o sufixo  $s[\ell:]$  em  $v$  ▷ desce a aresta inteira
11:   else
12:    crie  $w$  com rótulo  $\text{rótulo}(v)[1:\ell]$  ▷ split
13:    rebaixe  $v$  com o sufixo restante do rótulo
14:    se o sufixo restante for vazio, marque  $w$  como fim; senão crie folha irmã
15:   end if
16: end if
```

2.3 Decisões de projeto

1. Alfabeto da Trie/Patricia como dicionário (não vetor de 256 posições): adequado a strings curtas e ao experimento com prefixos latinos.
2. Patricia como radix tree compacta (rótulos string), e não Patricia binária bit a bit de Morrison — alinhada ao enunciado (“Radix Tree compacta”).
3. Splay recursivo em vez de *top-down* iterativo: o rastreamento visual das rotações fica mais direto (Seção 3).
4. Treap com heap *máximo* de prioridades em $[0, 1]$: um nó recém-inserido com prioridade alta sobe e ilustra o invariante.
5. KD-Tree genérica em k , com visualização 2D separada das retas de partição.
6. Instrumentação por objeto injetado, sem globais — cada corrida experimental começa com contadores zerados.

3 Demonstração e rastreamento visual

Os exemplos são pequenos de propósito: cada figura isola *um* mecanismo. As imagens foram geradas pelos scripts em *demos/* a partir do DOT exportado pela própria estrutura (matplotlib; **não** são figuras sintéticas de modelo generativo). Círculos duplos marcam fim de palavra.

3.1 Trie e Patricia: prefixo compartilhado versus compactação

As palavras *casa*, *caso*, *cama* e *carro* compartilham *c-a*. A Trie ramifica um caractere por aresta e *não* compacta o caminho *c-a-s-a*. A Patricia, ao inserir *caso* sobre *casa*, divide a aresta em *cas* + ramos *a* e *o* — exatamente o *split* do Algoritmo 1. Novas chaves redividem (*ca* + *s/m/rro*). A remoção de *carro* na Trie poda só o sufixo exclusivo; na Patricia, remover *caso* permite recompartar.



(a) Trie após casa, caso, cama, carro.

(b) Patricia: *split* do prefixo cas.

Figura 1: Mesmas chaves, organizações distintas: um símbolo por aresta (Trie) versus rótulo compacto (Patricia).

3.2 Splay e Treap: reorganização após operação

Inserções 10, 20, 30, 40, 50 na Splay deixam o último valor na raiz (cada inserção splaya). A busca de 10 aplica *zig-zig* na cadeia e coloca 10 no topo (Figura 2a). Na Treap, prioridades fixas no demo tornam a raiz a maior prioridade; inserir 25 com $p = 0,90$ provoca rotações até a raiz **sem** violar a ordem BST (Figura 2b). A remoção de 25 rotaciona o nó para baixo até extraí-lo.



(a) Splay: busca de 10 (sobe à raiz).

(b) Treap: 25 com $p = 0,90$ na raiz.

Figura 2: Dois mecanismos de reorganização: histórico de acesso (Splay) versus prioridade aleatória (Treap).

3.3 KD-Tree: partição espacial

Quatro pontos iniciais geram cortes verticais e horizontais alternados. Após remover (8,3), a consulta (6,5; 7,5) encontra (7,8) como vizinho mais próximo. O diagrama da árvore e os demais estados (remoção da Trie/Patricia/Splay) estão no repositório [Martins, 2026].

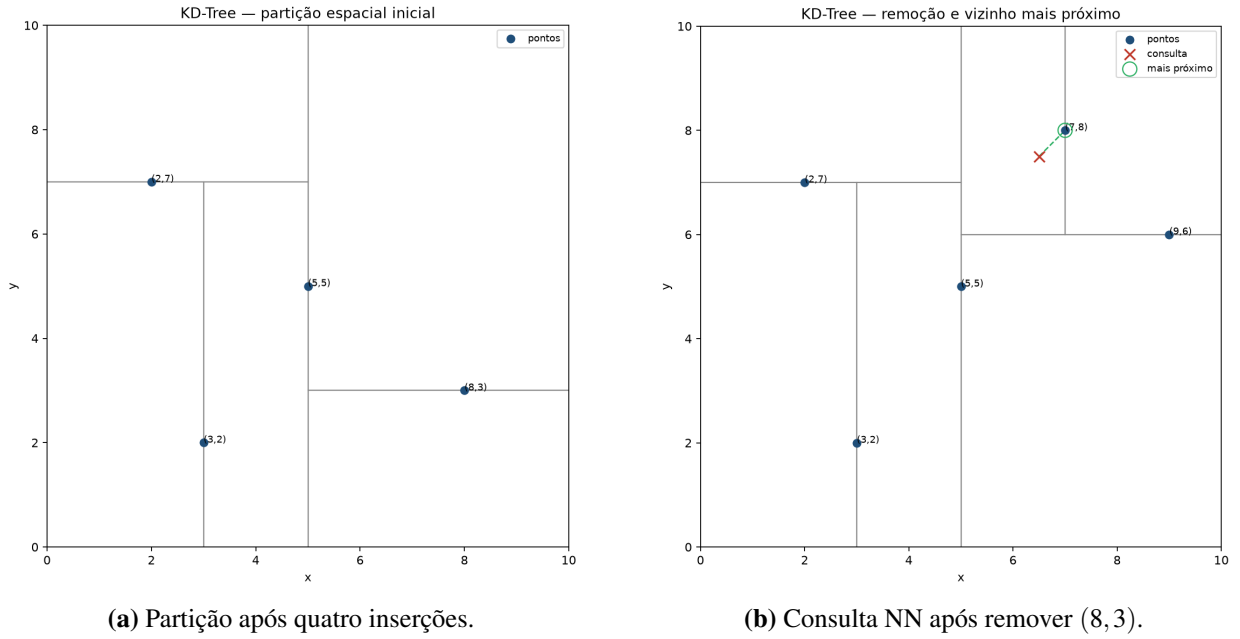


Figura 3: KD-Tree 2D: hiperplanos de corte e busca do vizinho mais próximo.

4 Análise de complexidade e comparação teórica

Seja n o número de chaves, L o comprimento da string, k a dimensão e m o número de pontos reportados numa *range query*. “Esperado” refere-se a prioridade ou ordem aleatória.

Tabela 2: Complexidade assintótica comparada às baselines BST e AVL.

Operação	Trie	Patricia	Splay	Treap	KD-Tree	BST	AVL
Busca (méd.)	$O(L)$	$O(L)$	$O(\log n)$ am.	$O(\log n)$ esp.	$O(n^{1-1/k})$	$O(\log n)$	$O(\log n)$
Busca (pior)	$O(L)$	$O(L)$	$O(n)$	$O(n)$	$O(n)$	$O(n)$	$O(\log n)$
Inserção (méd.)	$O(L)$	$O(L)$	$O(\log n)$ am.	$O(\log n)$ esp.	$O(\log n)$	$O(\log n)$	$O(\log n)$
Inserção (pior)	$O(L)$	$O(L)$	$O(n)$	$O(n)$	$O(n)$	$O(n)$	$O(\log n)$
Remoção (pior)	$O(L)$	$O(L)$	$O(n)$	$O(n)$	$O(n)$	$O(n)$	$O(\log n)$
Construção	$O(nL)$	$O(nL)$	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$
Específica	pref. $O(L+m)$	pref. $O(L+m)$	splay	—	NN / range	—	rot. $O(1)$
Espaço	$O(nL)$ pior	$O(n)$ nós típ.	$O(n)$	$O(n)$	$O(n)$	$O(n)$	$O(n)$

Por que esses custos. Na Trie cada símbolo é uma descida; o dicionário de filhos adiciona fator esperado $O(1)$ por nível. A Patricia percorre *caracteres* do rótulo, portanto permanece $O(L)$, mas visita menos nós — o ganho é espacial e de constante, não assintótico em L . Splay não limita a altura: uma sequência crescente deixa uma espinha; o potencial $\Phi = \sum \log s(v)$ mostra que a sequência de rotações custa $O(\log n)$ amortizado [Sleator and Tarjan, 1985]. Treap é BST aleatória: a probabilidade de um nó ser ancestral de outro é $O(1/|i-j|)$, e a altura esperada é $O(\log n)$ [Seidel and Aragon, 1996]; o pior caso (prioridades degeneradas) ainda existe, embora raro. KD-Tree em 2D tem NN esperado melhor que linear, mas um conjunto degenerado força visitar $\Theta(n)$ nós; *range report* é $O(\sqrt{n} + m)$ no plano para árvores bem construídas — aqui a árvore é incremental (não reconstruída pelo mediano). BST degenera em inserção ordenada; AVL restaura $|h_L - h_R| \leq 1$ com $O(1)$ rotações por inserção, daí o pior caso logarítmico [Cormen et al., 2012].

Comparação direta com BST/AVL: Splay compete com AVL quando há localidade e perde em acesso uniforme adverso. Treap aproxima AVL em esperança, com código mais curto e sem alturas. Trie/Patricia não substituem AVL para inteiros: o “ L ” de um inteiro em decimal é pequeno, mas o ganho de prefixo não existe. KD-Tree não é BST: não há percurso in-order global único. Inserção ordenada em $n = 10^4$ para BST e Splay foi omitida da bateria (custo quadrático e profundidade de recursão); o caso $n = 5 \cdot 10^3$ já demonstra a

degeneração.

5 Metodologia experimental e resultados

As cinco estruturas não operam sobre o mesmo tipo de chave. A metodologia usa **três grupos**, cada um com $n \in \{1000, 5000, 10000\}$, semente 20260919 e tempos medidos com `perf_counter`. Métricas internas (comparações, rotações, nós, `getsizeof` acumulado) acompanham o relógio, para separar custo assintótico de ruído do interpretador. O CSV completo está no repositório [Martins, 2026].

5.1 Grupo strings — Trie versus Patricia

Palavras sintéticas com prefixos compartilhados (`pre`, `pro`, `par`, ... + sufixo aleatório), cenário favorável à compactação. Operações: inserção das n palavras, busca das n palavras e consultas de prefixo. Hipótese: tempos da mesma ordem $O(nL)$, Patricia com **menos nós**.

Em $n = 10^4$, a Patricia terminou com **13 391 nós** contra **50 087** da Trie (cerca de $3,7\times$ menos; memória estimada 2,6 MB vs 10,5 MB). A inserção foi da mesma ordem (Patricia 0,035 s, Trie 0,043 s). A busca da Patricia foi *mais lenta* (0,036 s vs 0,012 s): o custo $O(L)$ permanece, mas cada nível compara um rótulo inteiro. Isso confirma a Seção 4: ganho **espacial**, não assintótico em tempo.

5.2 Grupo ordenável — Splay, Treap, BST e AVL

Padrões: (i) inserção aleatória; (ii) inserção ordenada $0..n-1$ (pior caso da BST); (iii) buscas Zipf e com janela de localidade após inserção aleatória. Hipótese: AVL estável; BST colapsa no ordenado; Splay se destaca em Zipf/localidade; Treap permanece próxima da AVL no aleatório.

Inserção aleatória em $n = 10^4$: AVL 0,100 s (6956 rotações), Treap 0,055 s (20293 rotações de heap), Splay 0,086 s (193 320 rotações), BST 0,042 s. A BST aleatória é barata porque não rebalanceia; a Splay paga cada acesso com rotações. Busca Zipf versus uniforme na Splay ($n = 10^4$): 0,036 s / 171 k comparações contra 0,074 s / 333 k — a localidade reduz trabalho, como o modelo amortizado prevê. Inserção **ordenada** em $n = 5 \cdot 10^3$: BST 2,58 s e 25 milhões de comparações (degeneração $\Theta(n^2)$); AVL 0,041 s; Treap 0,015 s. A Splay, nesse padrão, *insere* em 0,004 s (cada novo máximo vira raiz em $O(1)$), mas a busca seguinte custa 0,040 s.

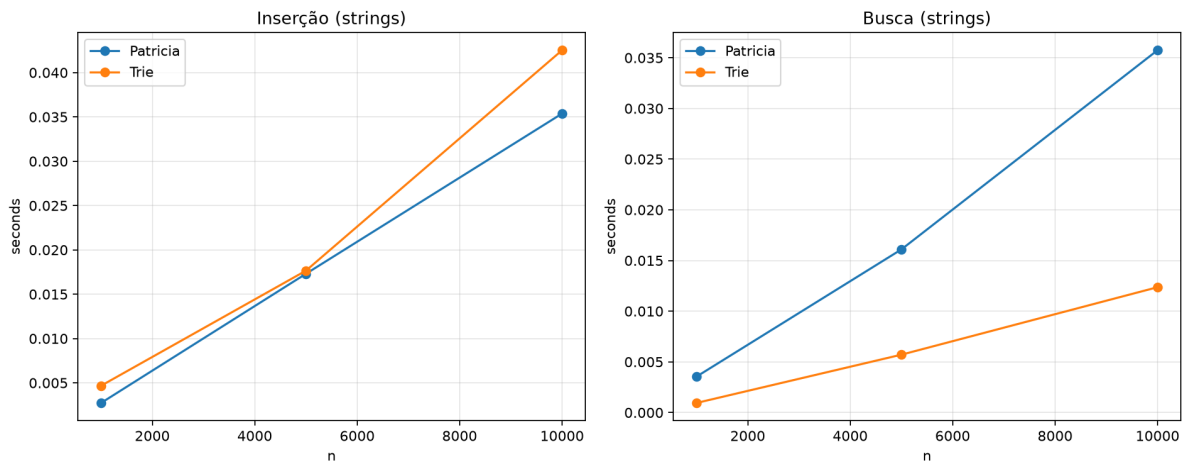
5.3 Grupo espacial — KD-Tree

Pontos uniformes em $[0, 1000]^k$, $k \in \{2, 3\}$. Inserção, 400 consultas de vizinho mais próximo contra força bruta $O(n)$ por consulta, e uma *range query* na caixa $[0, 250]^k$.

Em 2D, $n = 10^4$: KD-Tree 0,006 s versus força bruta 3,50 s ($\approx 580\times$). Em 3D o gap permanece (0,014 s vs 4,01 s), um pouco menor em vantagem relativa, coerente com a perda de poda quando k cresce. A construção incremental não impediu a poda neste conjunto uniforme.

5.4 Relação com a Seção 4

Os gráficos devem ser lidos contra a Tabela 2, não contra um “vencedor absoluto”. Python adiciona constante alta (dicionários, alocação). Divergências úteis: Trie e Patricia empatam ou invertem no relógio e se separam nos *nós*; Splay perde no acesso uniforme e recupera em Zipf; Treap acompanha AVL sem o pior caso rígido da BST; KD-Tree confirma a poda frente à varredura linear.

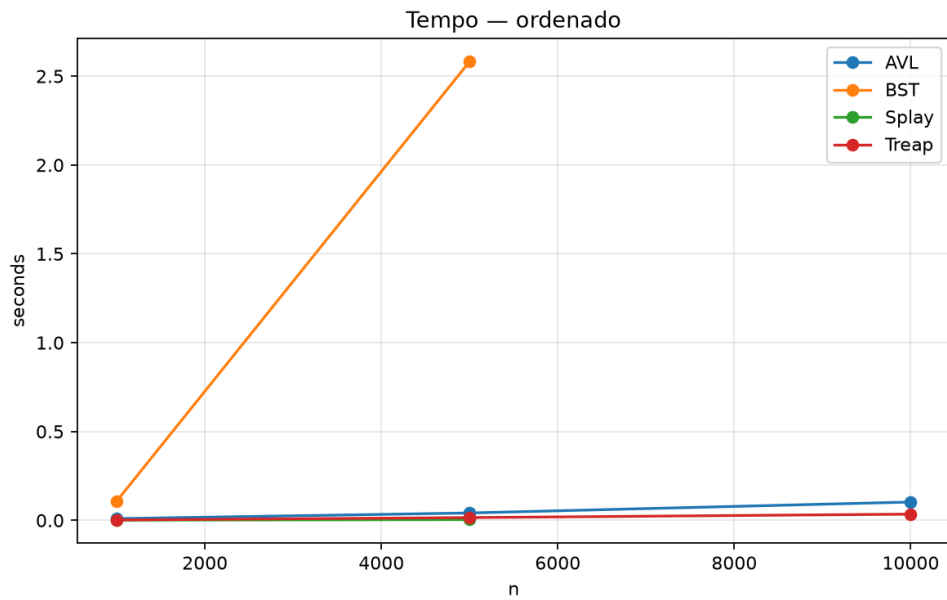


(a) Tempo de inserção e busca.

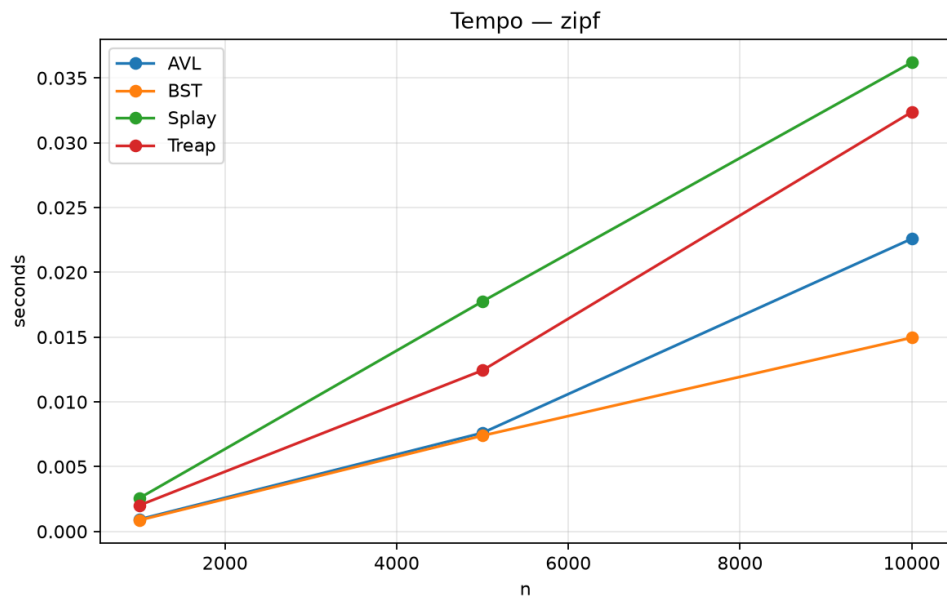


(b) Nós após a inserção.

Figura 4: Grupo strings. A Patricia empata (ou vence) na inserção e perde na busca, mas reduz drasticamente o número de nós.



(a) Inserção ordenada: BST degenera.



(b) Busca Zipf após inserção aleatória.

Figura 5: Grupo ordenável. A BST explode no padrão ordenado; a Splay reduz trabalho sob acesso enviesado.

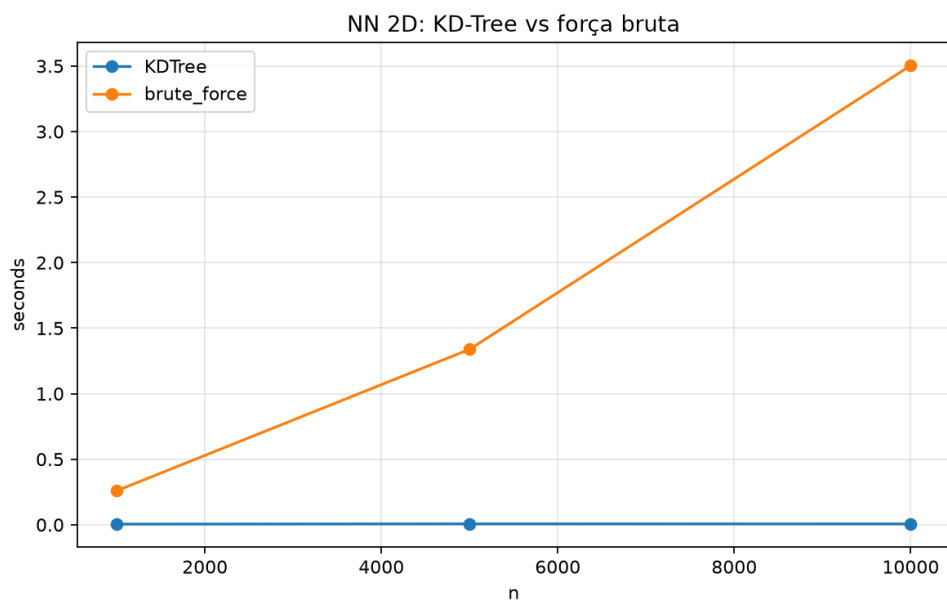


Figura 6: NN 2D (400 consultas): KD-Tree versus varredura linear.

6 Aplicações, análise crítica e discussão

Aplicações. **Trie:** autocompletar — a consulta é um prefixo e a enumeração das m sugestões é $O(L + m)$. **Patricia:** tabelas de roteamento IP e índices compactos; o *split/merge* de prefixos modela máscaras de rede melhor que uma Trie ingênua [Morrison, 1968]. **Splay:** caches e compressores (variantes de *move-to-front*); poucos documentos quentes justificam o custo das rotações [Sleator and Tarjan, 1985]. **Treap:** dicionário com implementação relativamente simples, ou treaps implícitas (extensão natural) [Seidel and Aragon, 1996]. **KD-Tree:** GIS, clustering preliminar e vizinhos em espaço de características de dimensão baixa ($k \leq 3$ neste trabalho) [Bentley, 1975].

Vantagens, limitações e esforço. A Trie é a mais simples e a mais gulosa em memória. A Patricia exige cuidado no *split* no meio da aresta e na compactação — foi o maior esforço de corretude (busca exata versus *starts_with* em rótulo parcial). Splay é curta de escrever, mas difícil de rastrear mentalmente (*zig-zag*). Treap foi a implementação mais limpa entre as BST-like. Na KD-Tree, remoção e NN com poda são o trecho mais delicado; a visualização 2D ajudou a validar o eixo de corte.

Limitações: a Patricia aqui não é bit a bit; a Splay não oferece pior caso determinístico; a Treap depende de gerador aleatório; a KD-Tree incremental não é ótima como a construída pelo mediano; memória via `sizeof` subestima objetos aninhados do interpretador (é comparativa, não absoluta). Melhorias possíveis: reconstruir a KD-Tree pelo mediano; Patricia binária; treap implícita; alfabeto compacto na Trie ASCII.

7 Conclusão

As cinco estruturas resolvem problemas diferentes, e a comparação só é justa *dentro* de cada universo de chaves.

1. **Diferenças centrais.** Trie e Patricia exploram prefixos; Splay explora o histórico de acesso; Treap explora aleatoriedade; KD-Tree explora geometria. BST/AVL exploram ordem total unidimensional.
2. **Desempenho.** Strings: tempos semelhantes, Patricia mais enxuta (13 k vs 50 k nós em $n = 10^4$). Ordenáveis: AVL previsível; Splay melhor sob Zipf; BST inaceitável na inserção ordenada (≈ 25 milhões de comparações em $n = 5 \cdot 10^3$); Treap como meio-termo. Espacial: KD-Tree $\approx 580\times$ mais rápida que força bruta em NN 2D.
3. **Coerência com a complexidade.** Onde o modelo distingue pior e médio (BST vs AVL, força bruta vs KD-Tree), o experimento reproduz a folga. Onde só mudam constantes (Trie vs Patricia em $O(L)$), o gráfico de nós informa mais que o de tempo.
4. **Implementação.** Patricia (*split/compactação*) e KD-Tree (remoção + poda) exigiram mais cuidado; Trie e Treap, menos.
5. **Escolha adequada.** Prefixo de strings $\rightarrow$ Patricia (espaço) ou Trie (simplicidade). Pior caso rígido $\rightarrow$ AVL. Carga enviesada $\rightarrow$ Splay. Balanceamento probabilístico simples $\rightarrow$ Treap. Pontos em $\mathbb{R}^2/\mathbb{R}^3 \rightarrow$ KD-Tree.
6. **Limites.** Nenhuma estrutura domina. Autoajuste, compactação, aleatoriedade e partição espacial são *estratégias*, não hierarquia de qualidade. A contribuição deste trabalho é tornar essas estratégias mensuráveis e visíveis, lado a lado com BST e AVL.

Referências

- Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein. *Algoritmos: teoria e prática*. Elsevier, Rio de Janeiro, 3 edition, 2012.
- Robert Sedgewick and Kevin Wayne. *Algorithms*. Addison-Wesley, Boston, 4 edition, 2011.
- Donald R. Morrison. PATRICIA — practical algorithm to retrieve information coded in alphanumeric. *Journal of the ACM*, 15(4):514–534, 1968. doi: 10.1145/321479.321481.
- Daniel Dominic Sleator and Robert Endre Tarjan. Self-adjusting binary search trees. *Journal of the ACM*, 32(3):652–686, 1985. doi: 10.1145/3828.3835.
- Raimund Seidel and Cecilia R. Aragon. Randomized search trees. *Algorithmica*, 16(4–5):464–497, 1996. doi: 10.1007/BF01940876.
- Jon Louis Bentley. Multidimensional binary search trees used for associative searching. *Communications of the ACM*, 18(9):509–517, 1975. doi: 10.1145/361002.361007.
- Kairo Henrique Ferreira Martins. Implementações, experimentos e figuras do trabalho prático i — estruturas em árvores avançadas. <https://github.com/KairoHenrique/TP1-Estruturas-em-Arvores-Avancadas>, 2026. Código-fonte em Python, demos visuais e CSV experimental.